

Q1. Smart University Payroll Management System

Score: 25.00 TC: 3/3 passed

CODE

```
import java.util.*;

class Main
{
    public static String readInput(Scanner sc)
    {
        String s = "";

        while(sc.hasNextLine())
        {
            s = sc.nextLine().trim();

            if(!s.isEmpty())
            {
                return s;
            }
        }

        return s;
    }

    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(readInput(sc));

        Employee emp[] = new Employee[n];

        for(int i=0;i<n;i++)
        {
            String type = readInput(sc);

            int id = Integer.parseInt(readInput(sc));
            String name = readInput(sc);
            String department = readInput(sc);
            double basicSalary = Double.parseDouble(readInput(sc));

            if(type.equalsIgnoreCase("TeachingFaculty"))
            {
                int teachingHours = Integer.parseInt(readInput(sc));
                double incentivePerHour = Double.parseDouble(readInput(sc));

                emp[i] = new TeachingFaculty(id,name,department,
                    basicSalary,teachingHours,incentivePerHour);
            }
            else if(type.equalsIgnoreCase("TechnicalStaff"))
            {
                int overtimeHours = Integer.parseInt(readInput(sc));
                double overtimeRate = Double.parseDouble(readInput(sc));

                emp[i] = new TechnicalStaff(id,name,department,
                    basicSalary,overtimeHours,overtimeRate);
            }
            else if(type.equalsIgnoreCase("VisitingFaculty"))
            {
                int lecturesHandled = Integer.parseInt(readInput(sc));
                double paymentPerLecture = Double.parseDouble(readInput(sc));

                emp[i] = new VisitingFaculty(id,name,department,
```

```
        basicSalary,lecturesHandled,paymentPerLecture);
    }
}

String search = readInput(sc);

if(search.equalsIgnoreCase("Search Employee ID:"))
{
    search = readInput(sc);
}

int searchId = Integer.parseInt(search);

Employee highest = emp[0];
double totalSalary = 0;

System.out.println("Employee Details");
System.out.println();

for(int i=0;i<n;i++)
{
    emp[i].displayEmployee();

    System.out.println("Monthly Salary : " + emp[i].calculateSalary());
    System.out.println();

    totalSalary += emp[i].calculateSalary();

    if(emp[i].calculateSalary() > highest.calculateSalary())
    {
        highest = emp[i];
    }
}

System.out.println("Highest Paid Employee");
System.out.println("Employee ID : " + highest.getEmployeeId());
System.out.println("Salary : " + highest.calculateSalary());
System.out.println();

System.out.println("Average Salary");

double avg = totalSalary / n;

if(avg == Math.floor(avg))
    System.out.println(avg);
else
    System.out.printf("%.2f\n",avg);

System.out.println();

boolean found = false;

for(int i=0;i<n;i++)
{
    if(emp[i].getEmployeeId()==searchId)
    {
        System.out.println("Employee Found");
        System.out.println("Employee ID : " + emp[i].getEmployeeId());
        System.out.println("Name : " + emp[i].getEmployeeName());
        System.out.println("Salary : " + emp[i].calculateSalary());

        found = true;
        break;
    }
}
```

```
    }

    if(!found)
    {
        System.out.println("Employee Not Found");
    }
}

class Employee
{
    private int employeeId;
    private String employeeName;
    private String department;
    private double basicSalary;

    public Employee()
    {

    }

    public Employee(int employeeId,String employeeName,
                    String department,double basicSalary)
    {
        this.employeeId=employeeId;
        this.employeeName=employeeName;
        this.department=department;
        this.basicSalary=basicSalary;
    }

    public int getEmployeeId()
    {
        return employeeId;
    }

    public String getEmployeeName()
    {
        return employeeName;
    }

    public String getDepartment()
    {
        return department;
    }

    public double getBasicSalary()
    {
        return basicSalary;
    }

    public double calculateSalary()
    {
        return basicSalary;
    }

    public void displayEmployee()
    {
        System.out.println("Employee ID : " + employeeId);
        System.out.println("Name : " + employeeName);
        System.out.println("Department : " + department);
    }
}

class TeachingFaculty extends Employee
{

```



```
private int teachingHours;
private double incentivePerHour;

public TeachingFaculty()
{

}

public TeachingFaculty(int employeeId,String employeeName,
                        String department,double basicSalary,
                        int teachingHours,double incentivePerHour)
{
    super(employeeId,employeeName,department,basicSalary);
    this.teachingHours = teachingHours;
    this.incentivePerHour = incentivePerHour;
}

@Override
public double calculateSalary()
{
    return getBasicSalary() + (teachingHours * incentivePerHour);
}
}

class TechnicalStaff extends Employee
{
    private int overtimeHours;
    private double overtimeRate;

    public TechnicalStaff()
    {

    }

    public TechnicalStaff(int employeeId,String employeeName,
                        String department,double basicSalary,
                        int overtimeHours,double overtimeRate)
    {
        super(employeeId,employeeName,department,basicSalary);
        this.overtimeHours = overtimeHours;
        this.overtimeRate = overtimeRate;
    }

    @Override
    public double calculateSalary()
    {
        return getBasicSalary() + (overtimeHours * overtimeRate);
    }
}

class VisitingFaculty extends Employee
{
    private int lecturesHandled;
    private double paymentPerLecture;

    public VisitingFaculty()
    {

    }

    public VisitingFaculty(int employeeId,String employeeName,
                        String department,double basicSalary,
                        int lecturesHandled,double paymentPerLecture)
    {
```

```
        super(employeeId,employeeName,department,basicSalary);
        this.lecturesHandled = lecturesHandled;
        this.paymentPerLecture = paymentPerLecture;
    }

    @Override
    public double calculateSalary()
    {
        return getBasicSalary() + (lecturesHandled * paymentPerLecture);
    }
}
```

INPUT

```
3

TeachingFaculty
101
Dr.Arun
CSE
60000
20
500

TechnicalStaff
102
Mr.Ravi
ECE
39500
15
300

VisitingFaculty
103
Mr.Kumar
IT
0
40
1200

Search Employee ID:
102
```

OUTPUT

```
Employee Details

Employee ID : 101
Name : Dr.Arun
Department : CSE
Monthly Salary : 60000.0

Employee ID : 102
Name : Mr.Ravi
Department : ECE
Monthly Salary : 39500.0

Employee ID : 103
Name : Mr.Kumar
Department : IT
Monthly Salary : 48000.0

Highest Paid Employee
Employee ID : 101
Salary : 60000.0

Average Salary
49166.67

Employee Found
Employee ID : 102
Name : Mr.Ravi
Salary : 39500.0
```

Q2. Smart Vehicle Insurance Claim System

Score: 25.00 TC: 3/3 passed

CODE

```
import java.util.*;

class Main
{
    public static String readInput(Scanner sc)
    {
        String s="";

        while(sc.hasNextLine())
        {
            s=sc.nextLine().trim();

            if(!s.isEmpty())
                return s;
        }

        return s;
    }

    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in);

        int n=Integer.parseInt(readInput(sc));

        Vehicle vehicle[]=new Vehicle[n];

        for(int i=0;i<n;i++)
        {
            String type=readInput(sc);

            String vehicleNumber=readInput(sc);
            String ownerName=readInput(sc);
            double insuredAmount=Double.parseDouble(readInput(sc));

            if(type.equalsIgnoreCase("Car"))
            {
                vehicle[i]=new Car(vehicleNumber,ownerName,insuredAmount);
            }
            else if(type.equalsIgnoreCase("Bike"))
            {
                vehicle[i]=new Bike(vehicleNumber,ownerName,insuredAmount);
            }
            else if(type.equalsIgnoreCase("Truck"))
            {
                vehicle[i]=new Truck(vehicleNumber,ownerName,insuredAmount);
            }
        }

        String search=readInput(sc);

        if(search.equalsIgnoreCase("Search Vehicle:"))
        {
            search=readInput(sc);
        }

        Vehicle highest=vehicle[0];

        for(int i=0;i<n;i++)
        {
```

```
        if(vehicle[i].calculateClaimAmount()>highest.calculateClaimAmount())
        {
            highest=vehicle[i];
        }
    }

    for(int i=0;i<n;i++)
    {
        System.out.println("Vehicle : " + vehicle[i].getVehicleNumber());
        System.out.println("Claim Amount : " + vehicle[i].calculateClaimAmount());
        System.out.println();
    }

    System.out.println("Highest Claim Vehicle");
    System.out.println(highest.getVehicleNumber());
    System.out.println(highest.calculateClaimAmount());
    System.out.println();

    boolean found=false;

    for(int i=0;i<n;i++)
    {
        if(vehicle[i].getVehicleNumber().equalsIgnoreCase(search))
        {
            System.out.println("Vehicle Found");
            System.out.println("Owner : " + vehicle[i].getOwnerName());
            System.out.println("Claim Amount : " + vehicle[i].calculateClaimAmount());

            found=true;
            break;
        }
    }

    if(!found)
    {
        System.out.println("Vehicle Not Found");
    }
}

class Vehicle
{
    private String vehicleNumber;
    private String ownerName;
    private double insuredAmount;

    public Vehicle()
    {

    }

    public Vehicle(String vehicleNumber,String ownerName,double insuredAmount)
    {
        this.vehicleNumber=vehicleNumber;
        this.ownerName=ownerName;
        this.insuredAmount=insuredAmount;
    }

    public String getVehicleNumber()
    {
        return vehicleNumber;
    }

    public String getOwnerName()
```



```
{
    return ownerName;
}

public double getInsuredAmount()
{
    return insuredAmount;
}

public double calculateClaimAmount()
{
    return insuredAmount;
}
}

class Car extends Vehicle
{
    public Car()
    {

    }

    public Car(String vehicleNumber,String ownerName,double insuredAmount)
    {
        super(vehicleNumber,ownerName,insuredAmount);
    }

    @Override
    public double calculateClaimAmount()
    {
        return getInsuredAmount() * 0.88;
    }
}

class Bike extends Vehicle
{
    public Bike()
    {

    }

    public Bike(String vehicleNumber,String ownerName,double insuredAmount)
    {
        super(vehicleNumber,ownerName,insuredAmount);
    }

    @Override
    public double calculateClaimAmount()
    {
        return getInsuredAmount() * 0.68;
    }
}

class Truck extends Vehicle
{
    public Truck()
    {

    }

    public Truck(String vehicleNumber,String ownerName,double insuredAmount)
    {
        super(vehicleNumber,ownerName,insuredAmount);
    }
}
```

Submissions Report

```
@Override
public double calculateClaimAmount()
{
    return getInsuredAmount() * 0.90;
}
}
```

INPUT

```
3

Car
TN10AB1234
Ramesh
800000

Bike
TN09XY5678
Suresh
100000

Truck
TN11TR9999
Logan
1500000

Search Vehicle:
TN09XY5678
```

OUTPUT

```
Vehicle : TN10AB1234
Claim Amount : 640000.0

Vehicle : TN09XY5678
Claim Amount : 60000.0

Vehicle : TN11TR9999
Claim Amount : 1350000.0

Highest Claim Vehicle
TN11TR9999
1350000.0

Vehicle Found
Owner : Suresh
Claim Amount : 60000.0
```